

Go. Chapter VIII: Goroutines and Channels

NEDO-OFFSEC

TABLE OF CONTENT

I	Goroutines	4
	1. Что это.....	4
	2. Пример: анимация во время вычислений.....	4
	3. Что здесь происходит.....	4
	4. Важно.....	4
	5. Синтаксис.....	4
	6. Ключевая мысль.....	5
II	EXAMPLE: Concurrent echo-server	6
	1. Простейший эхо-сервер (синхронный).....	6
	2. Эхо с задержкой (синхронный).....	6
	3. Параллельный эхо-сервер.....	6
	4. Клиент.....	6
	5. Важно!.....	6
	6. Ключевая мысль.....	6
III	Channels	7
	1. Что такое канал.....	7
	2. Основные операции.....	7
	3. Небуферизованный канал.....	7
	4. Буферизованный канал.....	7
	5. Заккрытие канала.....	7
	6. Однонаправленные каналы.....	7
	7. Конвейер (pipeline).....	7
	8. Итог.....	7
IV	Select	9
	1. Как работает.....	9
	2. Пример: таймер и отмена.....	9
	3. Таймеры.....	9
	4. Неблокирующий select (с default).....	9
	5. nil-каналы.....	9
	6. Ключевая мысль.....	10
V	Cancellation	11
	1. Проблема.....	11
	2. Решение: закрытие канала.....	11
	3. Проверка отмены.....	11
	4. Пример: отмена по нажатию Enter.....	11
	5. Обработка отмены в цикле.....	11
	6. Как это работает.....	11
	7. Ключевая мысль.....	11
VI	EXAMPLE: Chat Server	12
	1. Архитектура.....	12
	2. Каналы.....	12
	3. Broadcaster.....	12
	4. Обработка клиента.....	12
	5. Как это работает.....	13

6. Ключевая мысль..... 13

Go. Chapter VIII: Goroutines and Channels

Go обеспечивает два стиля параллельного программирования. В этой главе представлены go-подпрограммы (goroutines) и каналы, которые поддерживают взаимодействующие последовательные процессы (communicating sequential processes — CSP),

модель параллелизма, в которой между независимыми процессами (go-подпрограммами) передаются значения, но переменные по большей части ограничиваются одним процессом.

Goroutines

Что это

Горутина (goroutine) – это лёгкий поток выполнения в Go. Она запускается ключевым словом `go`.

```
f()      // обычный вызов -- ждём завершения
go f()   // запуск в новой горутине -- не ждём
```

При запуске программы есть одна горутина – `main`. Когда `main` завершается, все остальные горутин завершаются принудительно.

Пример: анимация во время вычислений

```
func main() {
    go spinner(100 * time.Millisecond) // запускаем анимацию в фоне
    fibN := fib(45)                    // долгое вычисление
    fmt.Printf("Fibonacci(%d) = %d\n", 45, fibN)
}

func spinner(delay time.Duration) {
    for {
        for _, r := range `-\|/` {
            fmt.Printf("\r%c", r)
            time.Sleep(delay)
        }
    }
}

func fib(x int) int {
    if x < 2 {
        return x
    }
    return fib(x-1) + fib(x-2)
}
```

Что здесь происходит

`go spinner(...)` запускает анимацию в фоне.

`fib(45)` выполняется одновременно.

Когда `main` завершится – программа заканчивается, даже если `spinner` ещё работает.

Важно

Горутин – это не потоки ОС. Они легче: стек растёт динамически, планировщик Go управляет ими.

Синтаксис

```
go func() {
    // код
}()
```

Анонимную функцию тоже можно запустить как горутину.

Ключевая мысль

`go` – это способ запустить функцию параллельно с основной программой. Программа завершается, когда завершается `main`, независимо от того, работают ли другие горутины.

EXAMPLE: Concurrent echo-server

Простейший эхо-сервер (синхронный)

```
func handleConn(c net.Conn) {
    io.Copy(c, c) // читает и сразу пишет обратно
    c.Close()
}
```

Эхо с задержкой (синхронный)

```
func echo(c net.Conn, shout string, delay time.Duration) {
    fmt.Fprintln(c, "\t", strings.ToUpper(shout))
    time.Sleep(delay)
    fmt.Fprintln(c, "\t", shout)
    time.Sleep(delay)
    fmt.Fprintln(c, "\t", strings.ToLower(shout))
}

func handleConn(c net.Conn) {
    input := bufio.NewScanner(c)
    for input.Scan() {
        echo(c, input.Text(), 1*time.Second) // синхронно
    }
    c.Close()
}
```

Проблема: пока обрабатывается один “крик”, новый ввод не принимается.

Параллельный эхо-сервер

Достаточно добавить `go` перед вызовом `echo`:

```
func handleConn(c net.Conn) {
    input := bufio.NewScanner(c)
    for input.Scan() {
        go echo(c, input.Text(), 1*time.Second) // асинхронно
    }
    c.Close()
}
```

Теперь каждый “крик” обрабатывается в своей горутине. Ответы приходят независимо друг от друга.

Клиент

```
func main() {
    conn, err := net.Dial("tcp", "localhost:8000")
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close()

    go mustCopy(os.Stdout, conn) // читаем ответ сервера
    mustCopy(conn, os.Stdin)    // отправляем ввод на сервер
}
```

Одна горутина читает ответы, другая отправляет запросы – параллельно.

Важно!

При вызове `go echo(...)` аргументы вычисляются в момент вызова, но сама функция запускается асинхронно.

Ключевая мысль

Одно слово `go` превращает синхронный сервер в параллельный. Горутины позволяют обрабатывать несколько соединений и несколько операций внутри одного соединения одновременно.

Channels

Что такое канал

Канал – это механизм связи между горутинами. Он позволяет передавать значения определённого типа.

```
ch := make(chan int) // канал для int
```

Основные операции

```
ch <- x    // отправка
x = <-ch   // получение
<-ch      // получение (игнорируем значение)
```

Небуферизованный канал

Отправка блокируется, пока кто-то не прочтает. Получение блокируется, пока кто-то не отправит.

```
ch := make(chan int)
go func() { ch <- 42 }()
x := <-ch // 42
```

Буферизованный канал

```
ch := make(chan string, 2)
ch <- "hello"
ch <- "world"
fmt.Println(<-ch) // hello
fmt.Println(<-ch) // world
```

Отправка блокируется только когда буфер заполнен. Получение блокируется когда буфер пуст.

Заккрытие канала

```
close(ch)
```

- После закрытия: чтение возвращает значение + ok == false
- Запись в закрытый канал вызывает панику

Однонаправленные каналы

```
var ch1 chan<- int // только отправка
var ch2 <-chan int // только получение
```

Конвейер (pipeline)

```
func main() {
    naturals := make(chan int)
    squares := make(chan int)

    go func() {
        for x := 0; x < 10; x++ {
            naturals <- x
        }
        close(naturals)
    }()

    go func() {
        for x := range naturals {
            squares <- x * x
        }
        close(squares)
    }()

    for x := range squares {
        fmt.Println(x)
    }
}
```

Итог

- Каналы – главный способ общения между горутинами.
- Небуферизованные – синхронные.
- Буферизованные – асинхронные (до заполнения буфера).

- Закрытие канала – сигнал, что данных больше не будет.

Select

`select` позволяет ждать несколько операций с каналами одновременно.

```
select {
case <-ch1:
    // ch1 готов
case x := <-ch2:
    // ch2 готов, x — полученное значение
case ch3 <- y:
    // можно отправить в ch3
default:
    // ничего не готово (неблокирующий режим)
}
```

Как работает

- `select` блокируется, пока один из каналов не станет готов.
- Если готово несколько — выбирает случайный.
- Если есть `default` — не блокируется.

Пример: таймер и отмена

```
func main() {
    abort := make(chan struct{})
    go func() {
        os.Stdin.Read(make([]byte, 1))
        abort <- struct{}{}
    }()

    fmt.Println("Обратный отсчёт. Enter — отмена.")
    select {
    case <-time.After(10 * time.Second):
        launch()
    case <-abort:
        fmt.Println("Отменено!")
    }
}
```

Таймеры

```
ticker := time.NewTicker(1 * time.Second)
defer ticker.Stop()
```

```
for countdown := 10; countdown > 0; countdown-- {
    fmt.Println(countdown)
    select {
    case <-ticker.C:
        // прошла секунда
    case <-abort:
        fmt.Println("Отменено!")
        return
    }
}
```

`time.Tick` удобен, но создаёт утечку горутины, если не остановить. Используйте `time.NewTicker` и `ticker.Stop()`.

Неблокирующий select (с default)

```
select {
case <-abort:
    fmt.Println("Отменено!")
    return
default:
    // ничего не делаем, продолжаем
}
```

nil-каналы

Операции с `nil`-каналом блокируются навсегда. Это позволяет включать/выключать варианты в `select`.

Ключевая мысль

`select` – это `switch` для каналов. Он позволяет ждать несколько событий одновременно и реагировать на первое готовое.

Cancellation

Проблема

Нет способа напрямую остановить горутину из другой горутины. Нужен механизм широковещания.

Решение: закрытие канала

Когда канал закрыт, все читающие получают нулевое значение. Это работает как широковещательный сигнал.

```
done := make(chan struct{}) // канал для сигнала отмены
close(done)                 // сигнал всем
```

Проверка отмены

```
func cancelled() bool {
    select {
    case <-done:
        return true
    default:
        return false
    }
}
```

Пример: отмена по нажатию Enter

```
done := make(chan struct{})

go func() {
    os.Stdin.Read(make([]byte, 1)) // ждём Enter
    close(done)                    // сигнал всем
}()

// В горутинах:
select {
case <-done:
    return // отмена
default:
    // продолжать работу
}
```

Обработка отмены в цикле

```
select {
case <-done:
    for range fileSizes {
        // опустошаем канал, чтобы горутины не заблокировались
    }
    return
case size, ok := <-fileSizes:
    // обработка размера файла
}
```

Как это работает

1. Создаём канал done типа chan struct{}
2. Закрываем его при отмене
3. Все горутины проверяют done в ключевых точках
4. При отмене – выходят из работы

Ключевая мысль

Закрытие канала – это способ отправить сигнал всем горутинам одновременно. Это дешевле и надёжнее, чем отправлять отдельные сообщения.

EXAMPLE: Chat Server

Архитектура

Чат-сервер использует 4 типа горутин:

1. `main` — принимает подключения
2. `broadcaster` — управляет клиентами и рассылает сообщения
3. `handleConn` — обрабатывает одного клиента
4. `clientWriter` — отправляет сообщения клиенту

Каналы

```
type client chan<- string // канал для отправки сообщений клиенту

var (
    entering = make(chan client) // новый клиент
    leaving  = make(chan client) // клиент уходит
    messages = make(chan string) // входящие сообщения
)
```

Broadcaster

```
func broadcaster() {
    clients := make(map[client]bool) // все подключённые клиенты

    for {
        select {
        case msg := <-messages:
            // отправить сообщение всем клиентам
            for cli := range clients {
                cli <- msg
            }
        case cli := <-entering:
            clients[cli] = true
        case cli := <-leaving:
            delete(clients, cli)
            close(cli)
        }
    }
}
```

Обработка клиента

```
func handleConn(conn net.Conn) {
    ch := make(chan string)
    go clientWriter(conn, ch)

    who := conn.RemoteAddr().String()
    ch <- "Вы " + who
    messages <- who + " подключился"
    entering <- ch

    input := bufio.NewScanner(conn)
    for input.Scan() {
        messages <- who + ": " + input.Text()
    }

    leaving <- ch
    messages <- who + " отключился"
    conn.Close()
}

func clientWriter(conn net.Conn, ch <-chan string) {
    for msg := range ch {
        fmt.Fprintln(conn, msg)
    }
}
```

Как это работает

1. `main` принимает подключения, запускает `handleConn`
2. `handleConn` создаёт канал для клиента, регистрирует его через `entering`
3. Клиент читает сообщения через `clientWriter`
4. Каждое сообщение отправляется в `messages`
5. `broadcaster` получает сообщение и отправляет его всем клиентам
6. При отключении клиент уведомляет через `leaving`

Ключевая мысль

Чат-сервер – это классический пример параллельного приложения, где горутины и каналы используются для управления множеством клиентов без явных блокировок.